

Vesti Transpiler User Manual

Sungbae Jeong

June 4, 2026

Contents

1	Introduction	1
2	Structure of Vesti File	2
3	Keywords	2
3.1	<code>docclass</code> keyword	3
3.2	<code>importpkg</code> keywords	3
3.3	<code>startdoc</code> keyword	3
3.4	<code>useenv</code> keyword	3
3.5	<code>beginv</code> and <code>endenv</code> keywords	4
3.6	<code>defun</code> keyword	4
4	Builtins	5
4.1	<code>#label</code> builtin	5
4.2	<code>#eq</code> builtin	5
4.3	<code>#chardef</code> builtin	6
4.4	<code>#mathchardef</code> builtin	6
4.5	<code>#enum</code> and <code>#enum_counter</code> builtins	6
4.6	<code>#get_filepath</code> builtin	7
4.7	<code>#at_on</code> and <code>#at_off</code> builtins	8
4.8	<code>#ltx3_on</code> and <code>#ltx3_off</code> builtins	8
4.9	<code>#textmode</code> and <code>#mathmode</code> builtins	8
5	Lua API	8
6	Source Code of This Document	9

1 Introduction

I used to create several documents using $\text{\LaTeX}$ (or plain $\text{\TeX}$ but $\text{\TeX}$ is quite cumbersome to write—especially when working with very complex tables or inserting images). Its markdown-like syntax is also not comfortable to use. For example, here is a simple $\text{\LaTeX}$ document:

```
1 % coprime is my custom class. See https://github.com/e0328eric/coprime.
2 \documentclass[tikz, geometry]{coprime}
3
4 \settitle{My First Document}{Sungbae Jeong}{}
5 \setgeometry{a4paper, margin = 2.5cm}
6
7 \begin{document}
8 \section{Foo}
9 Hello, World!
10 \begin{figure}[ht]
11   \centering
12   \begin{tikzpicture}
13     \draw (0,0) -- (1,1);
14   \end{tikzpicture}

```

```

15 \end{figure}
16
17 The code above is a figure using TikZ.
18
19 \end{document}

```

What annoys me most when using it is the ‘\begin’ and ‘\end’ blocks. Is there a way to write something much simpler? This question led me to start this project. Currently, the following code is generated into the $\text{\LaTeX}$ code above (except comments) using vesti:

```

1 % coprime is my custom class. See https://github.com/e0328eric/coprime.
2 \docclass coprime (tikz, geometry)
3
4 \settitle{My First Document}{Sungbae Jeong}{}
5 \setgeometry{a4paper, margin = 2.5cm}
6
7 \startdoc
8
9 \section{Foo}
10 Hello, World!
11 \useenv figure [ht] {
12     \centering
13     \useenv tikzpicture {
14         \draw (0,0) -- (1,1);
15     }
16 }
17
18 The code above is a figure using TikZ.

```

Everywhere in this paper, the symbol $\square$ denotes a “space” when one use Vesti.

2 Structure of Vesti File

Vesti is similar as $\text{\LaTeX}$. Its structure consists with two parts: **preamble** and **main**. Preamble is the place where $\text{\LaTeX}$ documentclass, packages, and several settings are located. Main body is where actual documentation is located. Below figure is the simple Vesti documentation.

```

1 \docclass article (10pt)
2 \importpkg {
3     geometry (a4paper, margin=2.2cm)
4 }
5 \startdoc
6 Hello, Vesti!

```

We will see later, but the very difference with $\text{\LaTeX}$ is that Vesti has its own keywords (keywords are colored with purple). It makes the code readable and it is easier and faster to write the document. The keyword `startdoc` splits the preamble and the main part of the documentation similar with

`\begin{document}` in $\text{\LaTeX}$. However, Vesti does not have the analogous part of `\end{document}`, because almost every $\text{\LaTeX}$ document (99.999% I’m sure) does not have any code below `\end{document}`. For this reason, Vesti automatically ends document when EOF (End Of File) is found.

3 Keywords

Followings are reserved as keywords. In this document, every Vesti keyword has the form like **this**.

<code>beginv</code>	<code>defenv</code>	<code>defun</code>	<code>docclass</code>
<code>endenv</code>	<code>importmod</code>	<code>importpkg</code>	<code>importves</code>
<code>startdoc</code>	<code>useenv</code>		

Table 1: Keywords in Vesti

3.1 `docclass` keyword

Keyword `docclass` is an analogous of `\documentclass` in $\text{\LaTeX}$. If `docclass` is in the main paragraph, it acts just a normal word. In other words, `docclass` activates only in the preamble. The syntax of `docclass` is following:

`docclass` $\square$ `<class name>` $\square$ `(<arguments>)`

Here, arguments are separated by commas and embraced by `()`. Here are some examples.

- `docclass` $\square$ `article`
- `docclass` $\square$ `article` $\square$ `(10pt)`
- `docclass` $\square$ `article` $\square$ `(10pt, \square twocols)`
- `docclass` $\square$ `article` $\square$ `(10pt,twocols)`

3.2 `importpkg` keywords

Keyword `importpkg` is an analogous of `\usepackage` in $\text{\LaTeX}$. If `importpkg` is in the main paragraph, it acts just a normal word. In other words, `importpkg` activates only in the preamble.

`importpkg` has two different syntax. First one is same as `docclass`.

`importpkg` $\square$ `<pkg-name>` $\square$ `(arguments)`

Here, arguments are separated by commas and embraced by `()`. In the practical case, one should include several packages with options. `importpkg` also supports such case. We will look at an example instead of giving rigorous grammar.

```

1 importpkg {
2   amsmath, amssymb, amsthm,
3   geometry (a4paper, margin=2.2cm),
4 }

```

As one can see, inside of `{}`, several packages can be used together with their options.

3.3 `startdoc` keyword

Keyword `startdoc` tells to Vesti that the main document starts. In the main document, you can also write `startdoc` in the main document. In that case, `startdoc` does nothing.

3.4 `useenv` keyword

As the name implies, keyword `useenv` is an analogous of `\begin{...}` and `\end{...}` pair in $\text{\LaTeX}$. The simplest `useenv` is like this.

`useenv` `center` {
Hello, World! or `useenv` `center` { Hello, World! }
}

As you can see, `useenv center` is the part of `\begin{center}`, and the single `}` is the part of `\end{center}`. Since Vesti knows their pair, one can write a code with several environment, and each pair is properly matched. For instance, above example is written in Vesti like follows. Here, `\useenv` just prints `useenv` in that style.

```

1 useenv figure [ht] {
2   \centering
3   useenv tikzpicture {
4     useenv scope {
5       \path (0,0) node {\vbox{
6         %#\hbox{\tt\useenv center \{}
7         %#\hbox{\tt\obeyspaces Hello, World!}
8         %#\hbox{\tt\obeyspaces\}}
9       }
10    }
11    \path (2.3,0) node {or};
12    useenv scope [shift={(6,0)}] {
13      \path (0,0) node {\tt\useenv center \{ Hello, World! \}};
14    }
15  }
16 }

```

Full syntax about **useenv** is the following.

useenv $\sqcup$ <environment name> $\sqcup$ <argument>* $\sqcup$ { <body> }

where “*” means that the number of <argument> is zero or at least one, and

$$\text{<argument>} = \begin{cases} (\text{<argument>}) & \text{mandatory arguments} \\ [\text{<argument>}] & \text{optional arguments} \end{cases}$$

For instance, below one is a valid Vesti code (environment **foo** is undefined in general). As one can see, spaces cannot exist in between <argument>s.

```

1 useenv foo (asd)(fff)[\ames and \awdsa](askws)[\rrsaa] {
2   foobar
3 }

```

3.5 **beginv** and **endenv** keywords

As the name implies, both keywords **beginv** and **endenv** are analogous of **\begin{...}** and **\end{...}** pair in $\text{\LaTeX}$, respectively. Thus below code

```

1 beginv center
2   asdsad
3 endenv

```

is exactly same as

```

1 useenv center {
2   asdsad
3 }

```

Then why we need **beginv** and **endenv** if we already have **useenv**? The only reason which both **beginv** and **endenv** exist is defining new environment. For instance, one defines a new environment like following.

```

1 defenv foo {beginv minipage(\textwidth)}{endenv}

```

3.6 **defun** keyword

defun is the keyword which makes a $\text{\LaTeX}$ function. Internally, it uses **xparse** package which is automatically included from modern $\text{\LaTeX}$ 2 ϵ kernel. The grammar of **defun** is the following.

defun $\sqcup$ [<attr>]? $\sqcup$ <command-name> $\sqcup$ (<param-spec>)? $\sqcup$ {<implementation>}

Here, `~?` means that this is an optional.

First, let us see some examples for `defun`, and then explain the meaning of each fields.

```
1 \defun foo {Hello, World!}
```

4 Builtin

Vesti also has its own builtin functions, which are prefixed with `#`. One might wonder what distinguishes builtins from keywords. In fact, from the compiler’s internal perspective, there is no real difference. However, in actual language usage, constantly typing the prefix can be somewhat tedious, especially for functions that are commonly used.

From the perspective of language design sometimes desirable to use names that can not be reserved as keywords. For example, Vesti provides a built-in function `#label`, which will be explained later. Since Vesti is a typewriting-oriented language, the word “label” is often used in its ordinary sense rather than in its special semantic meaning within the language.

Followings are reserved as builtin functions.

<code>#at_off</code>	<code>#at_on</code>	<code>#chardef</code>	<code>#copy_file</code>	<code>#def</code>
<code>#engine_type</code>	<code>#enum</code>	<code>#enum_counter</code>	<code>#eq</code>	<code>#get_filepath</code>
<code>#include</code>	<code>#label</code>	<code>#ltx3_off</code>	<code>#ltx3_on</code>	<code>#mathchardef</code>
<code>#mathmode</code>	<code>#noltx3</code>	<code>#picture</code>	<code>#raw_tex</code>	<code>#showfont</code>
<code>#textmode</code>	<code>#undef</code>			

Table 2: Builtins in Vesti

Since `fancyvrb` does not allow verbatim-like commands inside of `Verbatim` environment, builtins are not colored in example codes.

4.1 `#label` builtin

For the compiling issue, if one label some environment, one should write like follows using L^AT_EX function `\label`.

```
1 useenv theorem { \label{eq:1}
2   1 + 1 = 2.
3 }
```

However, such code is not natural. Thus `#label` comes into the place. One can label environments using `#label` builtin like this.

```
1 #label(eq:1) useenv theorem {
2   1 + 1 = 2.
3 }
```

One can also write like the following, which I personally prefer.

```
1 #label(eq:1)
2 useenv theorem {
3   1 + 1 = 2.
4 }
```

4.2 `#eq` builtin

`#eq` is a abbreviation of ‘`useenv` equation’. Actually, below codes are same.

```
1 #label(eq:1)
2 useenv equation {
3   \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
4 }
```

```

1 #eq(eq:1) {
2   \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
3 }

```

Since Vesti is used in the mathematical documents mainly, small syntactic sugar is needed, so Vesti introduces `#eq` builtin.

4.3 `#chardef` builtin

`#chardef` defines a text token using the unicode codepoint. Here is the grammar of `#chardef` builtin.

$$\text{\#chardef} \quad \langle \text{unicode-codepoint} \rangle \quad \langle \text{function} \rangle$$

Here, $\langle \text{unicode-codepoint} \rangle$ must be hexadecimal.

4.4 `#mathchardef` builtin

`#mathchardef` defines a math token using the unicode codepoint. Here is the grammar of `#mathchardef` builtin.

$$\text{\#mathchardef} \quad \langle \text{kind} \rangle \quad \langle \text{font-num} \rangle \quad \langle \text{unicode-codepoint} \rangle \quad \langle \text{function} \rangle$$

Here, $\langle \text{unicode-codepoint} \rangle$ must be hexadecimal. Before explaining each parameter, introduce some examples.

- `#mathchardef \opening 0 29d8 \lfoo`
- `#mathchardef \ordinary 0 2202 \diff`

4.5 `#enum` and `#enum_counter` builtins

`#enum` builtin is just an enumerate environment with minimal support of `enumitem` feature. For example,

```

1 #enum {
2   \item aaa
3   \item bbb
4   \item ccc
5 }

```

prints

1. aaa
2. bbb
3. ccc

`#enum` also can be nested like follows.

```

1 #enum {
2   \item aaa
3   \item #enum {
4     \item bbb
5     \item ccc
6   }
7   \item ddd
8 }

```

It prints

1. aaa
2. (a) bbb

(b) ccc

3. ddd

To customize a format of `\item` like `enumitem`, one can write like the following:

```
1 #enum (\Roman*. ) {  
2   \item aaa  
3   \item #enum ({\roman*}**) {  
4     \item bbb  
5     \item ccc  
6   }  
7   \item ddd  
8 }
```

I. aaa

II. i* bbb

ii* ccc

III. ddd

As one can see, `#enum` changes the format with inside of `()`. In Vesti, single `*` changes into `{<enum count>}` (including braces), and consecutive `**` changes into single `*` character. Thus, `\Roman*.` changes into `\Roman{enumi}.` and `{\roman*}**` changes into `{\roman{enumii}}*` (because this part is inside of another `#enum`).

Beware that by the parting mechanism, `\roman***` changes into `\roman*{<enum count>}`, and `\roman{*}**` into `\roman*{<enum count>}`, which are invalid arguments for `\roman` L^AT_EX internal command.

`#enum_counter` builtin is a helper to get the name of the current enum counter. For example, if one wants to start a second enum from 3, one can write

```
1 #enum {  
2   \item aaa  
3   \item #enum {  
4     \setcounter{#enum_counter}{2}  
5     \item bbb  
6     \item ccc  
7   }  
8   \item ddd  
9 }
```

and it prints

1. aaa

2. (c) bbb

(d) ccc

3. ddd

4.6 `#get_filepath` builtin

Because of the Vesti internal, if one includes picture by using `\includegraphics` for instance, providing the raw path cause an error from L^AT_EX with “file not found”. `#get_filepath` builtin adjust raw file path into the new relative one which L^AT_EX knows. Here is the example code.

```
1 \includegraphics{#get_filepath(./foo.png)}
```

Internally, `#get_filepath` changes file path into the relative one with respect to `.vesti-dummy`. For example, if `foo.png`, `foo.ves` and `.vesti-dummy` are located in the root directory, then inside of `foo.ves`, `#get_filepath(./foo.png)` will be changed into `../foo.png` because for `.vesti-dummy`, `foo.png` is located outside of it.

4.7 #at_on and #at_off builtins

Both builtins are same as L^AT_EX functions `\makeatletter` and `\makeatother` plus changes Vesti lexer such that `@` character is also can be a L^AT_EX function name. For instance, without using `#at_on` command, below code does not compiles.

```
1 defun @foo (#1) {Hello, #1!}
```

This is because `defun` expects a valid L^AT_EX function name but the character `@` is not a valid one except after the `\makeatletter` function. Since `defun` is a Vesti grammar, there is no way to tell Vesti that `@` is a right one although one uses `\makeatletter`. By using `#at_on`, below code then compiles.

```
1 #at_on
2 defun @foo (#1) {Hello, #1!}
3 #at_off
```

One can not using `#at_off`, but I strongly recommand to match pairs.

4.8 #ltx3_on and #ltx3_off builtins

Those builtins are similar with `#at_on` and `#at_off` pairs but for L^AT_EX3.

4.9 #textmode and #mathmode builtins

Internally, vesti tracks so called `text mode` and `math mode`. In the math mode, for instance, the token `->` is changed into `→` inside of the math mode. The idea is that Vesti changes `->` into `\to`, and so on. But when defining some function using `defun`, for example, one might want to define

```
1 useenv foo {-><-}
```

so that `$(foo)` makes `→←`, one should change the mode. Here, `#mathmode` comes into the place. One should write instead that

```
1 useenv foo {#mathmode{-><-}}
```

5 Lua API

Vesti uses Lua in both building script and inline inside of Vesti code. Below are functions which are baked in Vesti compiler.

compile	download	getCurrentDir	getEngineType
getModule	joinpath	mkdir	parse
ping	print	setCurrentDir	unzip
vestiDummyDir			

Table 3: Lua builtin functions in Vesti

6 Source Code of This Document

Below code was generated by inline lua.

```
1 docclass article (10pt)
2 importpkg {
3     geometry (a4paper, margin = 2.2cm),
4     xcolor,
5     tikz,
6     fancyvrb,
7     amsmath,
8 }
9
10 \title{Vesti Transpiler User Manual}
11 \author{Sungbae Jeong}
12 #::
13 local function read_all(path)
14     local f, err = io.open(path, "rb")
15     assert(f, ("cannot open %s: %s"):format(path, err))
16     local data = f:read("*a")
17     f:close()
18     return data
19 end
20 ::#<readAll>
21
22 -- definition of \keyword command
23 defun [!] keyword (m) {{\tt\color{purple}#1}}
24 defun [!] builtin (v) {{\tt\color{blue!65!yellow}\##1}}
25 #::
26 local commands = { "useenv", "defun", "defenv" }
27 for _, command in ipairs(commands) do
28     local s = string.format(
29         "defun %s (s) {{\IfBooleanTF{#1}{\keyword{%%!-%s-%%}}{\keyword{%%!-%s-%%} } }",
30         command, command, command
31     )
32     vesti.print(vesti.parse(s))
33 end
34 ::#
35
36 #::
37 function begenv(add_bang, add_space)
38     local name = "begenv"
39     if add_bang then name = name .. "!" end
40     local begenv = "\\keyword{" .. name .. "}"
41     if add_space then begenv = begenv .. " " end
42     vesti.print(begenv, { nl = 0 })
43 end
44
45 function endenv(add_bang, add_space)
46     local name = "endenv"
47     if add_bang then name = name .. "!" end
48     local endenv = "\\keyword{" .. name .. "}"
49     if add_space then endenv = endenv .. " " end
50     vesti.print(endenv, { nl = 0 })
51 end
52 ::#*
53
54 -- space character used in the textbook.
55 #at_on
56 #chardef 2423 \@b
```

```

57 defun ob {\kern2pt{\tt\@b}\kern2pt}
58 #at_off
59
60 startdoc
61 \maketitle
62 \tableofcontents
63
64 \section{Introduction}
65 I used to create several documents using \LaTeX\ (or plain \TeX\ but \TeX\ is
66 quite cumbersome to write-especially when working with very complex tables or
67 inserting images). Its markdown-like syntax is also not comfortable to use. For
68 example, here is a simple \LaTeX\ document:
69
70 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
71 % coprime is my custom class. See https://github.com/e0328eric/coprime.
72 \documentclass[tikz, geometry]{coprime}
73
74 \settitle{My First Document}{Sungbae Jeong}{ }
75 \setgeometry{a4paper, margin = 2.5cm}
76
77 \begin{document}
78 \section{Foo}
79 Hello, World!
80 \begin{figure}[ht]
81   \centering
82   \begin{tikzpicture}
83     \draw (0,0) -- (1,1);
84   \end{tikzpicture}
85 \end{figure}
86
87 The code above is a figure using TikZ.
88
89 \end{document}
90 -%}
91
92 What annoys me most when using it is the \lq\verb@\begin@\rq\ and
93 \lq\verb@\end@\rq\ blocks. Is there a way to write something much simpler? This
94 question led me to start this project. Currently, the following code is
95 generated into the \LaTeX code above (except comments) using vesti:
96
97 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
98 % coprime is my custom class. See https://github.com/e0328eric/coprime.
99 +keyword|docclass@ coprime (tikz, geometry)
100
101 \settitle{My First Document}{Sungbae Jeong}{ }
102 \setgeometry{a4paper, margin = 2.5cm}
103
104 +keyword|startdoc@
105
106 \section{Foo}
107 Hello, World!
108 +keyword|useenv@ figure [ht] {
109   \centering
110   +keyword|useenv@ tikzpicture {
111     \draw (0,0) -- (1,1);
112   }
113 }
114
115 The code above is a figure using TikZ.
116 -%}

```

```

117
118 Everywhere in this paper, the symbol \ob\ denotes a \lq\lq space\rq\rq\ when
119 one use Vesti.
120
121 \section{Structure of Vesti File}
122 Vesti is similar as \LaTeX. Its structure consists with two parts: {\tt preamble} and
123 {\tt main}. Preamble is the place where \LaTeX\ documentclass, packages, and
124 several settings are located. Main body is where actual documentation is located.
125 Below figure is the simple Vesti documentation.
126
127 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
128 +keyword|docclass@ article (10pt)
129 +keyword|importpkg@ {
130     geometry (a4paper, margin=2.2cm)
131 }
132 +keyword|startdoc@
133 Hello, Vesti!
134 -%}
135
136 We will see later, but the very difference with \LaTeX\ is that Vesti has its
137 own keywords (keywords are colored with purple). It makes the code readable and
138 it is easier and faster to write the document. The keyword startdoc splits
139 the preamble and the main part of the documentation similar with
140
141 -- Don't ask why I chose Q for catcode 0.
142 %#{\tt\catcode`Q=0 Qcatcode`\=12 \beginQ{documentQ}} in \LaTeX.
143 However, Vesti does not have the analogous part of
144 %#{\tt\catcode`Q=0 Qcatcode`\=12 \endQ{documentQ}},
145 because almost every \LaTeX\ document (99.999\% I'm sure) does not have any code
146 below %#{\tt\catcode`Q=0 Qcatcode`\=12 \endQ{documentQ}}.
147 For this reason, Vesti automatically ends document when EOF (End Of File) is
148 found.
149
150 \section{Keywords}
151 Followings are reserved as keywords. In this document, every Vesti keyword has
152 the form like \keyword{this}.
153 useenv table [ht] {
154     \centering
155     #::
156     local content = read_all("../src/lexer/token.rs")
157
158     -- Keywords live in `lookup_keyword` as `"name" => TokenType::Variant`.
159     -- We capture both the keyword and the variant, then filter out the
160     -- deprecated ones (which map to `TokenType::Deprecated`).
161     -- Pattern breakdown:
162     -- "([~"]+)"          => a quoted keyword -> capture 1
163     -- %s*=>%s*TokenType:: => the match arm to a TokenType variant
164     -- ([%w_]+)          => the variant name -> capture 2
165     local pat = '"([~"]+)"%s*=>%s*TokenType::([%w_]+)'
166
167     local keywords = {}
168     for name, tok in content:gmatch(pat) do
169         if tok ~= "Deprecated" then
170             keywords[#!keywords + 1] = name
171         end
172     end
173
174     table.sort(keywords)
175
176     vesti.print([\begin{tabular}{cccc}])

```

```

177
178   for i, kw in ipairs(keywords) do
179       local cell = string.format("\\keyword{%s}", kw)
180       if (i % 4) == 0 then
181           vesti.print(cell .. [[\\]])
182       else
183           vesti.print(cell .. "&")
184       end
185   end
186
187   vesti.print([[\\end{tabular}]]))
188   ::#[readAll]
189   \\caption{Keywords in Vesti}
190 }
191
192 \\subsection{\\keyword{docclass} keyword}
193 Keyword \\keyword{docclass} is an analogous of \\verb|\\documentclass| in \\LaTeX.
194 If \\keyword{docclass} is in the main paragraph, it acts just a normal word.
195 In other words, \\keyword{docclass} actives only in the preamble.
196 The syntax of \\keyword{docclass} is following:
197
198 useenv center {
199     \\keyword{docclass}\\ob<class name>\\ob{\\tt{<arguments>}{\\tt}}
200 }
201 Here, arguments are separated by commas and embraced by {\\tt ()}. Here are some
202 examples.
203
204 \\goodbreak
205 useenv itemize {
206     \\item \\keyword{docclass}\\ob{\\tt article}
207     \\item \\keyword{docclass}\\ob{\\tt article\\ob(10pt)}
208     \\item \\keyword{docclass}\\ob{\\tt article\\ob(10pt,\\ob twocols)}
209     \\item \\keyword{docclass}\\ob{\\tt article\\ob(10pt,twocols)}
210 }
211
212 \\subsection{\\keyword{importpkg} keywords}
213 Keyword \\keyword{importpkg} is an analogous of \\verb|\\usepackage| in \\LaTeX.
214 If \\keyword{importpkg} is in the main paragraph, it acts just a normal word.
215 In other words, \\keyword{importpkg} actives only in the preamble.
216
217 importpkg has two different syntax. First one is same as docclass.
218 useenv center {
219     \\keyword{importpkg}\\ob<pkg-name>\\ob{\\tt{arguments}{\\tt}}
220 }
221 Here, arguments are separated by commas and embraced by {\\tt ()}.
222 In the practical case, one should include several packages with options.
223 importpkg also supports such case. We will look at an example instead of
224 giving rigorous grammar.
225 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
226 +keyword|importpkg@ {
227     amsmath, amssymb, amsthm,
228     geometry (a4paper, margin=2.2cm),
229 }
230 -%}
231
232 \\noindent As one can see, inside of \\verb|{}|, several packages can be used
233 together with thier options.
234
235 \\subsection{\\keyword{startdoc} keyword}
236 Keyword \\keyword{startdoc} tells to Vesti that the main document starts. In the

```

```

237 main document, you can also write \keyword{startdoc} in the main document. In
238 that case, \keyword{startdoc} does nothing.
239
240 \subsection{\useenv keyword}
241 As the name implies, keyword \useenv is an analogous of \verb|\begin{...}| and
242 \verb|\end{...}| pair in \LaTeX.
243 The simplest \useenv is like this.
244
245 useenv figure [ht] {
246     \centering
247     useenv tikzpicture {
248         useenv scope {
249             \path (0,0) node {\vbox{
250                 %#\hbox{\tt\useenv center \{ }
251                 %#\hbox{\tt\obeyspaces Hello, World!}
252                 %#\hbox{\tt\obeyspaces\}}
253             } };
254         }
255         \path (2.3,0) node {or};
256         useenv scope [shift={(6,0)}]{
257             \path (0,0) node {\tt\useenv center \{ Hello, World! \}};
258         }
259     }
260 }
261
262 As you can see, {\tt\useenv center} is the part of \verb|\begin{center}|, and
263 the single {\tt\}} is the part of \verb|\end{center}|. Since Vesti knows their
264 pair, one can write a code with several environment, and each pair is properly
265 matched. For instance, above example is written in Vesti like follows. Here,
266 \verb|\useenv| just prints \useenv in that style.
267
268 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
269 |+color|purple@useenv@ figure [ht] {
270     \centering
271     |+color|purple@useenv@ tikzpicture {
272         |+color|purple@useenv@ scope {
273             \path (0,0) node {\vbox{
274                 |+color|blue@%#\hbox{\tt\useenv center \{ }
275                 |+color|blue@%#\hbox{\tt\obeyspaces Hello, World!}
276                 |+color|blue@%#\hbox{\tt\obeyspaces\}}
277             } };
278         }
279         \path (2.3,0) node {or};
280         |+color|purple@useenv@ scope [shift={(6,0)}] {
281             \path (0,0) node {\tt\useenv center \{ Hello, World! \}};
282         }
283     }
284 }
285 -%}
286
287 Full syntax about \useenv is the following.
288 useenv center {
289     \useenv\ob<environment name>\ob<argument>*\ob{\tt\{ } <body> {\tt\}}
290 }
291 where `*' means that the number of <argument> is zero or at least one, and
292 $$
293     "<argument>" = useenv cases {
294         "<argument>" & "mandatory arguments" \cr
295         "[<argument>]" & "optional arguments" \cr
296     }

```

```

297 $$
298 For instance, below one is a valid Vesti code (environment {\tt foo} is
299 undefined in general). As one can see, spaces cannot exist in between <argument>s.
300 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
301 |+color|purple@useenv@ foo (asd)(fff)[\ames and \awdsa](askws)[\rrsaa] {
302     foobar
303 }
304 -%}
305
306 #def #begenv {#::begenv(false,true)::#*}
307 #def #begenv_arg {#::begenv(#1,#2)::#*}
308 #def #endenv {#::endenv(false,true)::#*}
309 #def #endenv_arg {#::endenv(#1,#2)::#*}
310
311 \subsection{#begenv and #endenv keywords}
312 As the name implies, both keywords #begenv and #endenv are
313 analogous of \verb|\begin{...}| and \verb|\end{...}| pair in \LaTeX,
314 respectively. Thus below code
315 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
316 |+color|purple@begenv@ center
317     asdsad
318 |+color|purple@endenv@
319 -%}
320 is exactly same as
321 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
322 +keyword|useenv@ center {
323     asdsad
324 }
325 -%}
326 Then why we need #begenv and #endenv if we already have \useenv*?
327 The only reason which both #begenv and #endenv exist is
328 defining new environment. For instance, one defines a new environment like
329 following.
330 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@] {
331 %#+keyword|defenv@ foo { |+color|purple@begenv@ minipage(\textwidth)}{ |+color|purple@endenv@}
332 }
333
334 \subsection{\defun keyword}
335 \defun is the keyword which makes a \LaTeX\ function. Internally, it uses {\tt
336 xparse} package which is automatically included from modern \LaTeX$2\epsilon$
337 kernel. The grammar of \defun is the following.
338
339 begenv center
340     \defun\ob[<attr>]? \ob<command-name> \ob(<param-spec>)? \ob\{<implementation>\}
341 endenv
342 Here, {\tt`?`} means that this is an optional.
343
344 First, let us see some examples for \defun*, and then explain the meaning of
345 each fields.
346
347 #::
348 local examples = {
349     "\defun foo {Hello, World!}"
350 }
351
352 for _, example in ipairs(examples) do
353     vesti.print(vesti.parse(
354         "begenv! Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]"
355     ))
356     vesti.print(example)

```

```

357     vesti.print(vesti.parse("endenv! Verbatim"))
358 end
359 ::#
360
361 \section{Builtins}
362 Vesti also has its own builtin functions, which are prefixed with \#.
363 One might wonder what distinguishes builtins from keywords. In fact, from the
364 compiler's internal perspective, there is no real difference. However, in actual
365 language usage, constantly typing the prefix can be somewhat tedious, especially
366 for functions that are commonly used.
367
368 From the perspective of language design --particularly in Vesti-- it is
369 sometimes desirable to use names that can not be reserved as keywords. For example,
370 Vesti provides a built-in function \builtin|label|, which will be explained
371 later. Since Vesti is a typewriting-oriented language, the word \lq\lq
372 label\rq\rq\ is often used in its ordinary sense rather than in its special
373 semantic meaning within the language.
374
375 Followings are reserved as builtin functions.
376
377 useenv table [ht] {
378     \centering
379     #::
380     local content = read_all("../src/lexer/token.rs")
381
382     -- Builtins are the bare string literals inside the `builtin_set!` lists,
383     -- i.e. lines of the form `    "name",`. Keyword arms like
384     -- `"docclass" => TokenType::Docclass` have more text after the string, so
385     -- the end-of-line anchor below excludes them.
386     local builtins = {}
387     for line in content:gmatch("[^\r\n]+") do
388         local name = line:match('^%s*"([%w_]+)",?%s*$')
389         if name then
390             builtins[#!builtins + 1] = name
391         end
392     end
393     table.sort(builtins)
394
395     vesti.print([[ \begin{tabular}{ccccc} ]])
396
397     for i, kw in ipairs(builtins) do
398         local cell = string.format("\\builtin@%s@", kw)
399         if (i % 5) == 0 then
400             vesti.print(cell .. [[ \\ ]])
401         else
402             vesti.print(cell .. "&")
403         end
404     end
405
406     vesti.print([[ \\end{tabular} ]])
407     ::#[readAll]
408     \caption{Builtins in Vesti}
409 }
410 Since {\tt fancyvrb} does not allow verbatim-like commands inside of
411 {\tt Verbatim} environment, builtins are not colored in example codes.
412
413 \subsection{\builtin|label| builtin}
414 For the compiling issue, if one label some environment, one should write like
415 follows using \LaTeX\ function \verb|\label|.
416 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=&|@]{%-

```

```

417 &keyword|useenv@ theorem { \label{eq:1}
418     1 + 1 = 2.
419 }
420 -%}
421
422 However, such code is not natural. Thus \builtin|label| comes into the place.
423 One can label environments using \builtin|label| builtin like this.
424 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=&|@]{%-
425 #label(eq:1) &keyword|useenv@ theorem {
426     1 + 1 = 2.
427 }
428 -%}
429 One can also write like the following, which I personally prefer.
430 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=&|@]{%-
431 #label(eq:1)
432 &keyword|useenv@ theorem {
433     1 + 1 = 2.
434 }
435 -%}
436
437 \subsection{\builtin|eq| builtin}
438 \builtin|eq| is a abbreviation of \lq\keyword{%-useenv-%} equation\rq.
439 Actually, below codes are same.
440
441 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
442 #label(eq:1)
443 +keyword|useenv@ equation {
444     \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
445 }
446 -%}
447 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
448 #eq(eq:1) {
449     \sum_{n=1}^{\infty} {1//n^2} = {\pi^2//6}.
450 }
451 -%}
452 Since Vesti is used in the mathematical documents mainly, small syntactic suger
453 is needed, so Vesti introduce \builtin|eq| builtin.
454
455 \subsection{\builtin|chardef| builtin}
456 \builtin|chardef| defines a text token using the unicode codepoint.
457 Here is the grammar of \builtin|chardef| builtin.
458 useenv center {
459     \builtin|chardef|\ob<unicode-codepoint>\ob<function>
460 }
461 Here, <unicode-codepoint> must be hexadecimal.
462
463 \subsection{\builtin|mathchardef| builtin}
464 \builtin|mathchardef| defines a math token using the unicode codepoint.
465 Here is the grammar of \builtin|mathchardef| builtin.
466 useenv center {
467     \builtin|mathchardef|\ob<kind>\ob<font-num>\ob<unicode-codepoint>\ob<function>
468 }
469 Here, <unicode-codepoint> must be hexadecimal.
470 Before explaining each parameter, introduce some examples.
471 useenv itemize {
472     \item \builtin|mathchardef|\ob.opening\ob0\ob29d8\ob\verb|\lfoo|
473     \item \builtin|mathchardef|\ob.ordinary\ob0\ob2202\ob\verb|\diff|
474 }
475
476 \subsection{\builtin|enum| and \builtin|enum_counter| builtins}

```

```

477 \builtin|enum| builtin is just an enumerate environment with minimal support of
478 {\tt enumitem} feature. For example,
479 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
480 #enum {
481     \item aaa
482     \item bbb
483     \item ccc
484 }
485 -%}
486 prints
487 #enum {
488     \item aaa
489     \item bbb
490     \item ccc
491 }
492 \builtin|enum| also can be nested like follows.
493 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
494 #enum {
495     \item aaa
496     \item #enum {
497         \item bbb
498         \item ccc
499     }
500     \item ddd
501 }
502 -%}
503 It prints
504 #enum {
505     \item aaa
506     \item #enum {
507         \item bbb
508         \item ccc
509     }
510     \item ddd
511 }
512 To customize a format of \verb|\item| like {\tt enumitem}, one can
513 write like the following:
514 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
515 #enum (\Roman*. ) {
516     \item aaa
517     \item #enum ({\roman*}**) {
518         \item bbb
519         \item ccc
520     }
521     \item ddd
522 }
523 -%}
524 #enum (\Roman*. ) {
525     \item aaa
526     \item #enum ({\roman*}**) {
527         \item bbb
528         \item ccc
529     }
530     \item ddd
531 }
532 As one can see, \builtin|enum| changes the format with inside of {\tt()}.
533 In Vesti, single {\tt *} changes into {\tt \{<enum count>\}} (including braces),
534 and consecutive {\tt **} changes into single {\tt *} character.
535 Thus, \verb|\Roman*.| changes into \verb|\Roman{enumi}.| and \verb|{\roman*}**|
536 changes into \verb|{\roman{enumii}}*| (because this part is inside of another

```

```

537 \builtin|enum|).
538
539 Beware that by the parting mechanism, \verb|\roman***| changes into
540 \verb|\roman*{<enum count>}|, and \verb|\roman{*}**| into \verb|\roman{{<enum count>}}*|,
541 which are invalid arguments for \verb|\roman| \LaTeX\ internal command.
542
543 \builtin|enum_counter| builtin is a helper to get the name of the current enum
544 counter. For example, if one wants to start a second enum from 3, one can write
545 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
546 #enum {
547     \item aaa
548     \item #enum {
549         \setcounter{#enum_counter}{2}
550         \item bbb
551         \item ccc
552     }
553     \item ddd
554 }
555 -%}
556 and it prints
557 #enum {
558     \item aaa
559     \item #enum {
560         \setcounter{#enum_counter}{2}
561         \item bbb
562         \item ccc
563     }
564     \item ddd
565 }
566
567 \subsection{\builtin|get_filepath| builtin}
568 Because of the Vesti internal, if one includes picture by using
569 \verb|\includegraphics| for instance, providing the raw path cause an error from
570 \LaTeX\ with \lq\lq file not found\rq\rq. \builtin|get_filepath| builtin adjust
571 raw file path into the new relative one which \LaTeX\ knows. Here is the example
572 code.
573 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@] {
574 %#\includegraphics{#get_filepath{./foo.png}}
575 }
576 Internally, \builtin|get_filepath| changes file path into the relative one with
577 respect to {\tt.vesti-dummy}. For example, if {\tt foo.png}, {\tt foo.ves}
578 and {\tt.vesti-dummy} are located in the root directory, then inside of {\tt
579 foo.ves}, \builtin|get_filepath|{\tt{./foo.png}} will changed into
580 {\tt{../foo.png}} because for {\tt.vesti-dummy}, {\tt foo.png} is located outside
581 of it.
582
583 \subsection{\builtin|at_on| and \builtin|at_off| builtins}
584 Both builtins are same as \LaTeX\ functions \verb|\makeatletter| and
585 \verb|\makeatother| plus changes Vesti lexer such that \verb|@| character is
586 also can be a \LaTeX\ function name. For instance, without using \builtin|at_on|
587 command, below code does not compiles.
588 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|&] {
589 %#+keyword|defun& @foo (#1) {Hello, #1!}
590 }
591 This is because \keyword{% -defun-%} expects a valid \LaTeX\ function name but
592 the character \verb|@| is not a valid one except after the \verb|\makeatletter|
593 function. Since \keyword{% -defun-%} is a Vesti grammar, there is no way to tell
594 Vesti that \verb|@| is a right one although one uses \verb|\makeatletter|.
595 By using \builtin|at_on|, below code then compiles.
596 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|&]{%-

```

```

597 #at_on
598 +keyword|defun& @foo (#1) {Hello, #1!}
599 #at_off
600 -%}
601 One can not using \builtin|at_off|, but I strongly recommend to match
602 pairs.
603
604 \subsection{\builtin|ltx3_on| and \builtin|ltx3_off| builtins}
605 Those builtins are similar with \builtin|at_on| and \builtin|at_off|
606 pairs but for \LaTeX3.
607
608 \subsection{\builtin|textmode| and \builtin|mathmode| builtins}
609 Internally, vesti tracks so called {\tt text mode} and {\tt math mode}.
610 In the math mode, for instance, the token \verb|>| is changed into $->$ inside
611 of the math mode. The idea is that Vesti changes \verb|>| into \verb|\to|, and
612 so on. But when defining some function using \defun*, for example, one might
613 want to define
614 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
615 +keyword|useenv@ foo {-><-}
616 -%}
617 so that \verb|$ \foo$| makes $-><-$, one should change the mode. Here,
618 \builtin|mathmode| comes into the place. One should write instead that
619 useenv Verbatim [numbers=left, numbersep=5pt, frame=single, commandchars=+|@]{%-
620 +keyword|useenv@ foo {#mathmode{-><-}}
621 -%}
622
623 \section{Lua API}
624 Vesti uses Lua in both building script and inline inside of Vesti code.
625 Below are functions which are baked in Vesti compiler.
626 useenv table [ht] {
627     \centering
628     #::
629     local content = read_all("../src/lua.rs")
630
631     -- Each `vesti.*` function is registered with `table.set("<name>", ...)`
632     local pat = 'table%.set%("([~"]+)"'
633
634     local lua = {}
635     for name in content:gmatch(pat) do
636         lua[#lua + 1] = name
637     end
638     table.sort(lua)
639
640     vesti.print([[\\begin{tabular}{ccccc}]]
641
642     for i, kw in ipairs(lua) do
643         local cell = string.format("\\tt{%s}", kw)
644         if (i % 4) == 0 then
645             vesti.print(cell .. [[\\]])
646         else
647             vesti.print(cell .. "&")
648         end
649     end
650
651     vesti.print([[\\end{tabular}]])
652     ::#[readAll]
653     \caption{Lua builtin functions in Vesti}
654 }
655
656

```

```
657 \newpage
658 \section{Source Code of This Document}
659 Below code was generated by inline lua.
660 useenv Verbatim [numbers=left, numbersep=5pt, frame=single] {
661 #::
662     local content = read_all("vesti_man.ves")
663     for line in content:gmatch("([^\r\n]*)\r?\n?") do
664         vesti.print(line)
665     end
666 ::#[readAll]
667 }
668
```